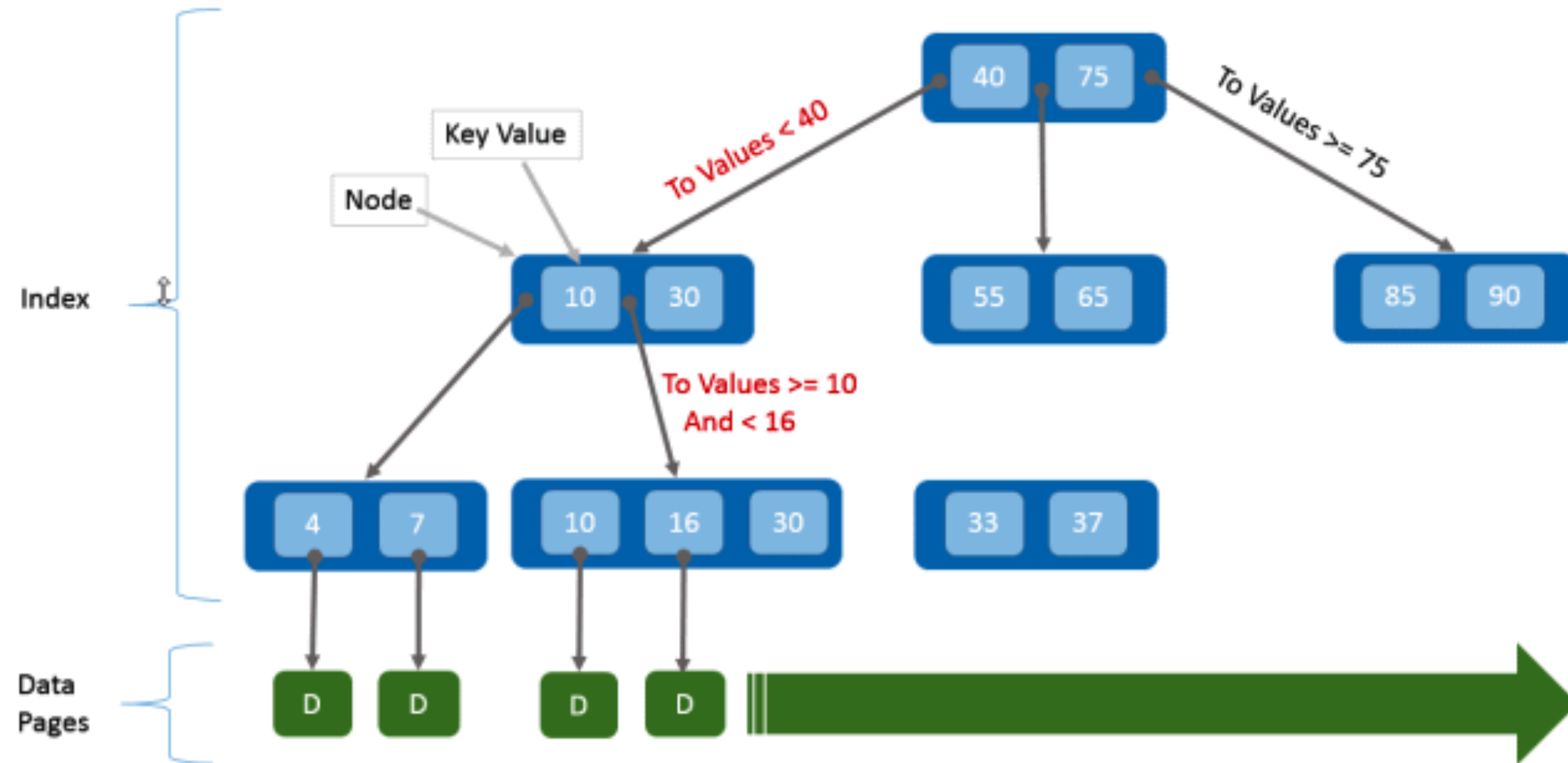


**Индексы, партиционирование
и сегментирование.**

Индексы

B-Tree Layout



Индекс – это объект базы данных, обеспечивающий дополнительные способы быстрого поиска и извлечения данных.

Плюсы:

- При наличии индекса время поиска нужных строк можно существенно уменьшить.

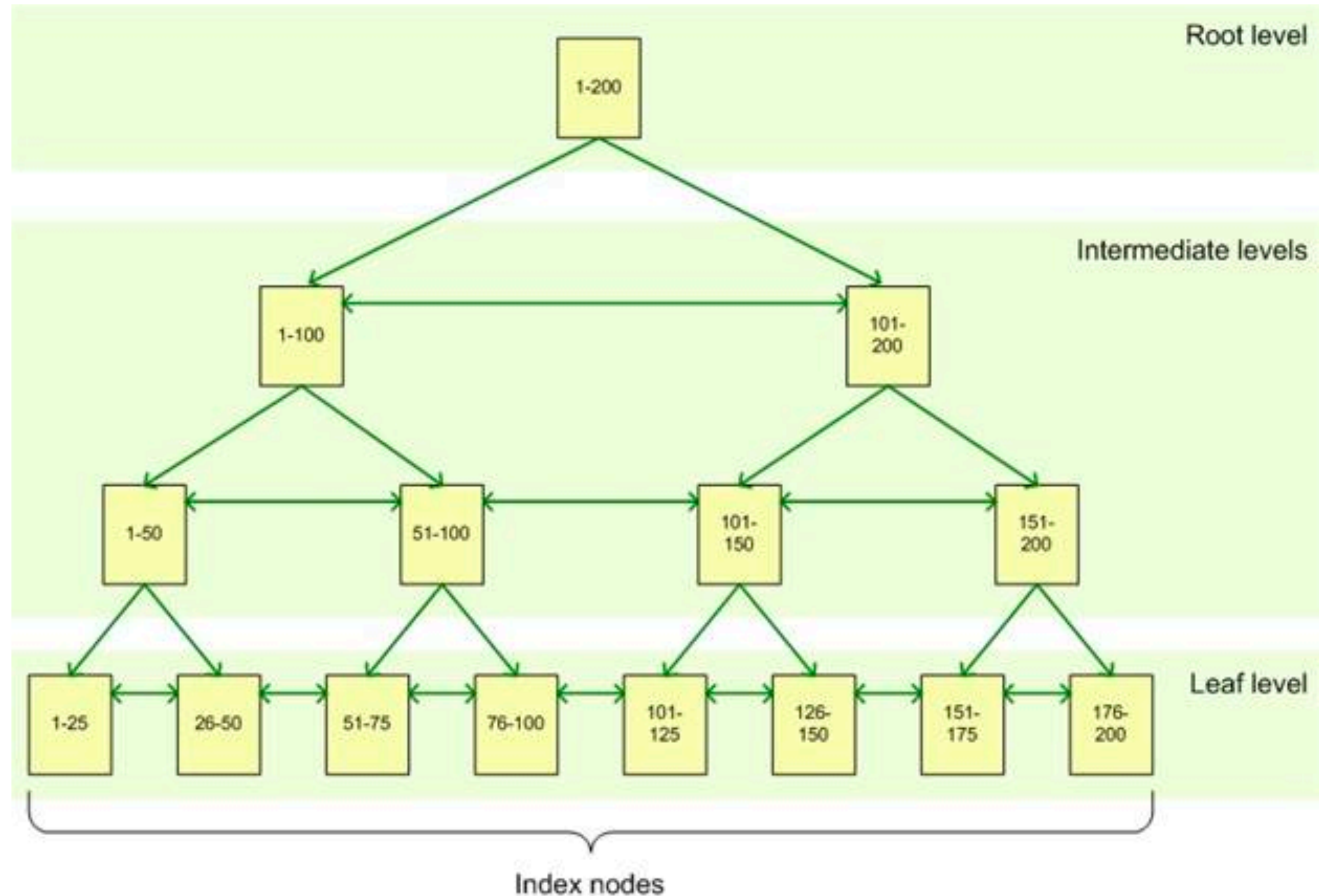
Минусы:

- Дополнительное место на диске и в оперативной памяти,
- Замедляются операции вставки, обновления и удаления записей.

Классификация индексов

Существует два типа индексов:

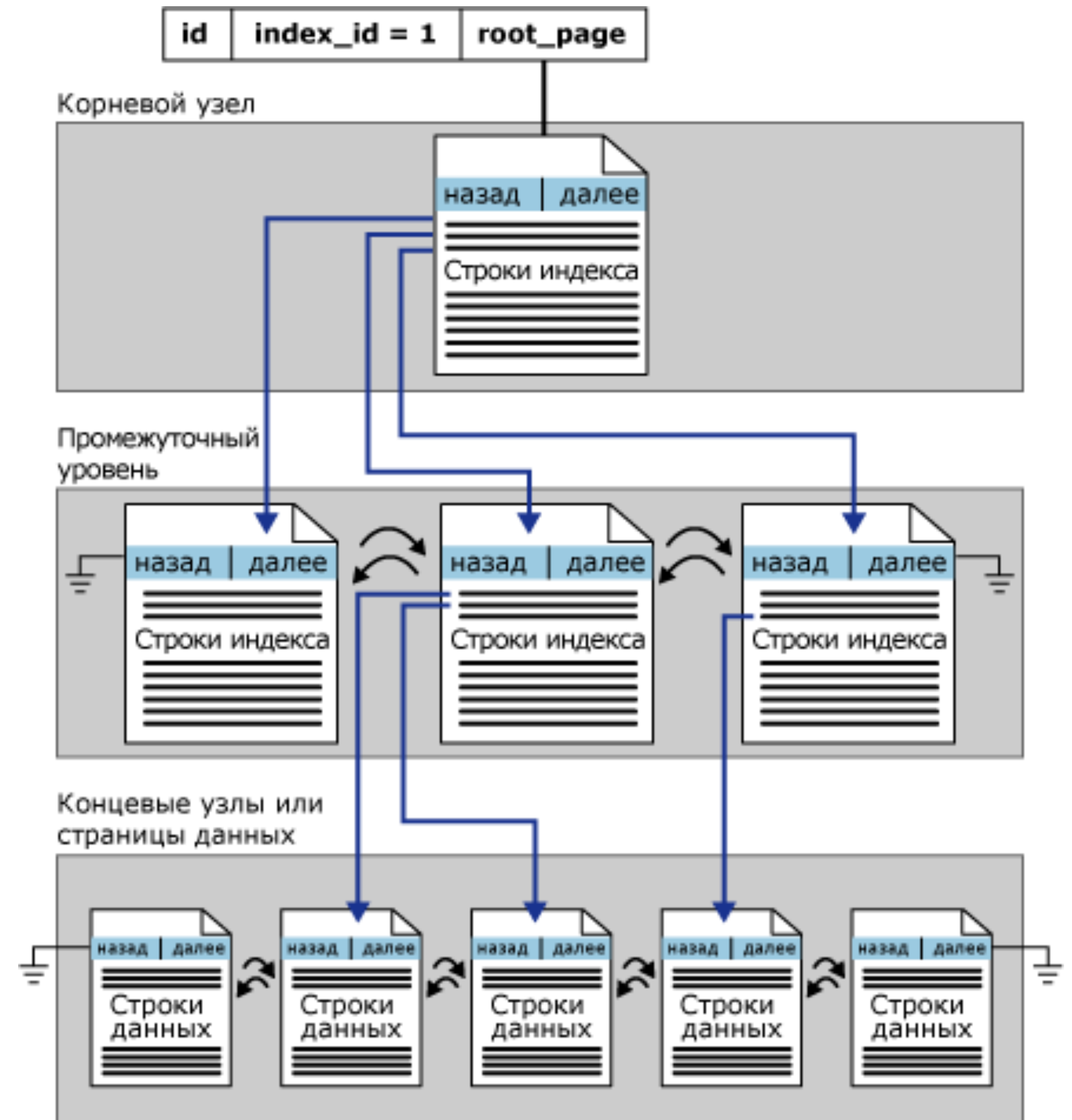
- Кластерные индексы;
- Некластерные индексы, которые включают:
 - на основе кучи;
 - на основе кластерных индексов.



Кластерные индексы

В кластерном индексе таблица представляет собой часть индекса, или индекс представляет собой часть таблицы.

- Листовой узел кластерного индекса – это страница таблицы с данными.
- Поскольку сами данные таблицы являются частью индекса, то очевидно, что для таблицы может быть создан только один кластерный индекс.
- Кластерный индекс является уникальным индексом по определению. Это означает, что все ключи записей должны быть уникальные.



Пример создания индекса на основе В-дерева

Подведем итог:

- В-tree (сбалансированное дерево) — это самый распространенный тип индекса в PostgreSQL.
- Он поддерживает все стандартные операции сравнения ($>$, $<$, $>=$, $<=$, $=$, $<>$) и может использоваться с большинством типов данных.
- В-tree индексы могут быть использованы для сортировки, ограничений уникальности и поиска по диапазону значений.

Описание инструкции:

```
CREATE INDEX ИМЯ  
ON таблица  
[USING тип индекса] (столбец);
```

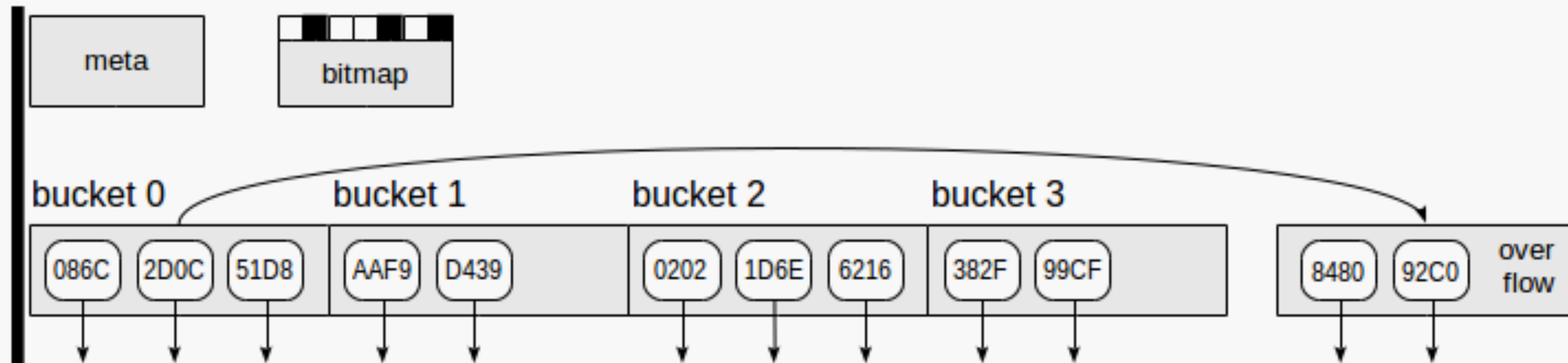
Пример создания индекса:

```
CREATE INDEX ix_example_btree  
ON example_table (column_name);
```

Какие еще бывают индексы

- **Хеш-индексы** хранят 32-битный хеш-код, полученный из значения индексируемого столбца, поэтому хеш-индексы работают только с простыми условиями равенства. Планировщик запросов может применить хеш-индекс, только если индексируемый столбец участвует в сравнении с оператором.
- **GIN-индексы** представляют собой «инвертированные индексы», в которых могут содержаться значения с несколькими ключами, например массивы. Инвертированный индекс содержит отдельный элемент для значения каждого компонента, и может эффективно работать в запросах, проверяющих присутствие определённых значений компонентов.
- **GiST-индексы** представляют собой не просто разновидность индексов, а инфраструктуру, позволяющую реализовать много разных стратегий индексирования. Как следствие, GiST-индексы могут применяться с разными операторами, в зависимости от стратегии индексирования (*класса операторов*).
- **BRIN-индексы** (сокращение от Block Range INdexas, Индексы зон блоков) хранят обобщённые сведения о значениях, находящихся в физически последовательно расположенных блоках таблицы. Поэтому такие индексы наиболее эффективны для столбцов, значения в которых хорошо коррелируют с физическим порядком столбцов таблицы.

Хеш-индексы



Хеш-индекс использует четыре типа страниц (серые прямоугольники):

- Meta page — это нулевая страница, содержащая информацию о том, что находится внутри индекса.
- Bucket pages — это основные страницы индекса, на которых хранятся данные в виде пар «хэш-код — TID».
- Overflow pages — структурированы так же, как страницы корзины, и используются, когда одной страницы корзины недостаточно.
- Bitmap pages — это страницы с переполнением, которые в данный момент свободны и могут быть повторно использованы для других сегментов.

Стрелки вниз, начинающиеся у элементов индексной страницы, обозначают TID, то есть ссылки на строки таблицы.

GIN-индексы

Основная область применения метода GIN — ускорение полнотекстового поиска, поэтому его целесообразно использовать в качестве примера при более подробном обсуждении этого индекса.

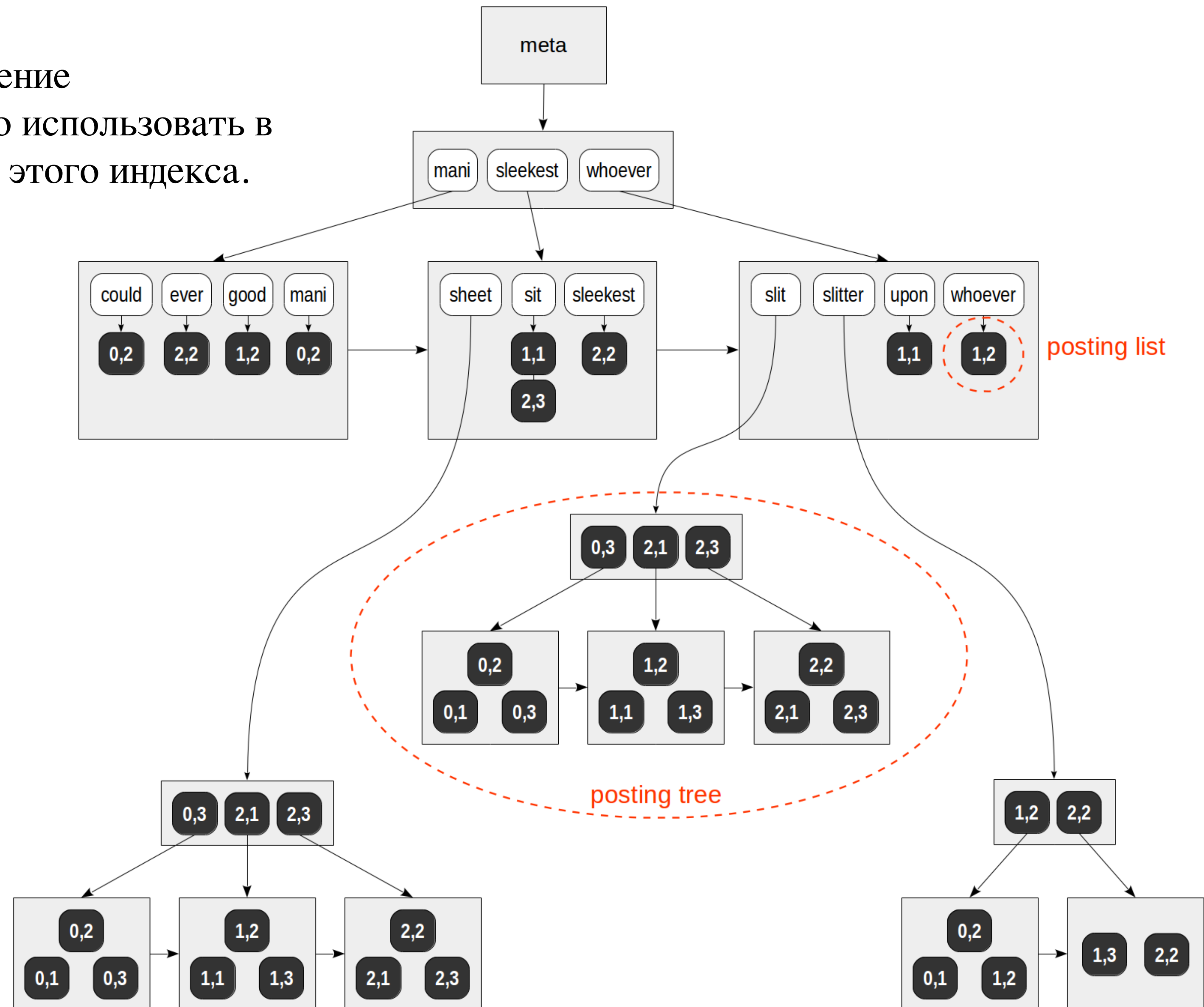
```
create table ts(doc text, doc_tsv tsvector);
```

```
insert into ts(doc) values
```

```
('Can a sheet slitter slit sheets?'),  
( 'How many sheets could a sheet slitter slit?'),  
( 'I slit a sheet, a sheet I slit.'),  
( 'Upon a slitted sheet I sit.'),  
( 'Whoever slit the sheets is a good sheet slitter.'),  
( 'I am a sheet slitter.'),  
( 'I slit sheets.'),  
( 'I am the sleekest sheet slitter that ever slit sheets.'),  
( 'She slits the sheet she sits on.');
```

```
update ts set doc_tsv = to_tsvector(doc);
```

```
create index test_index on ts using gin(doc_tsv);
```

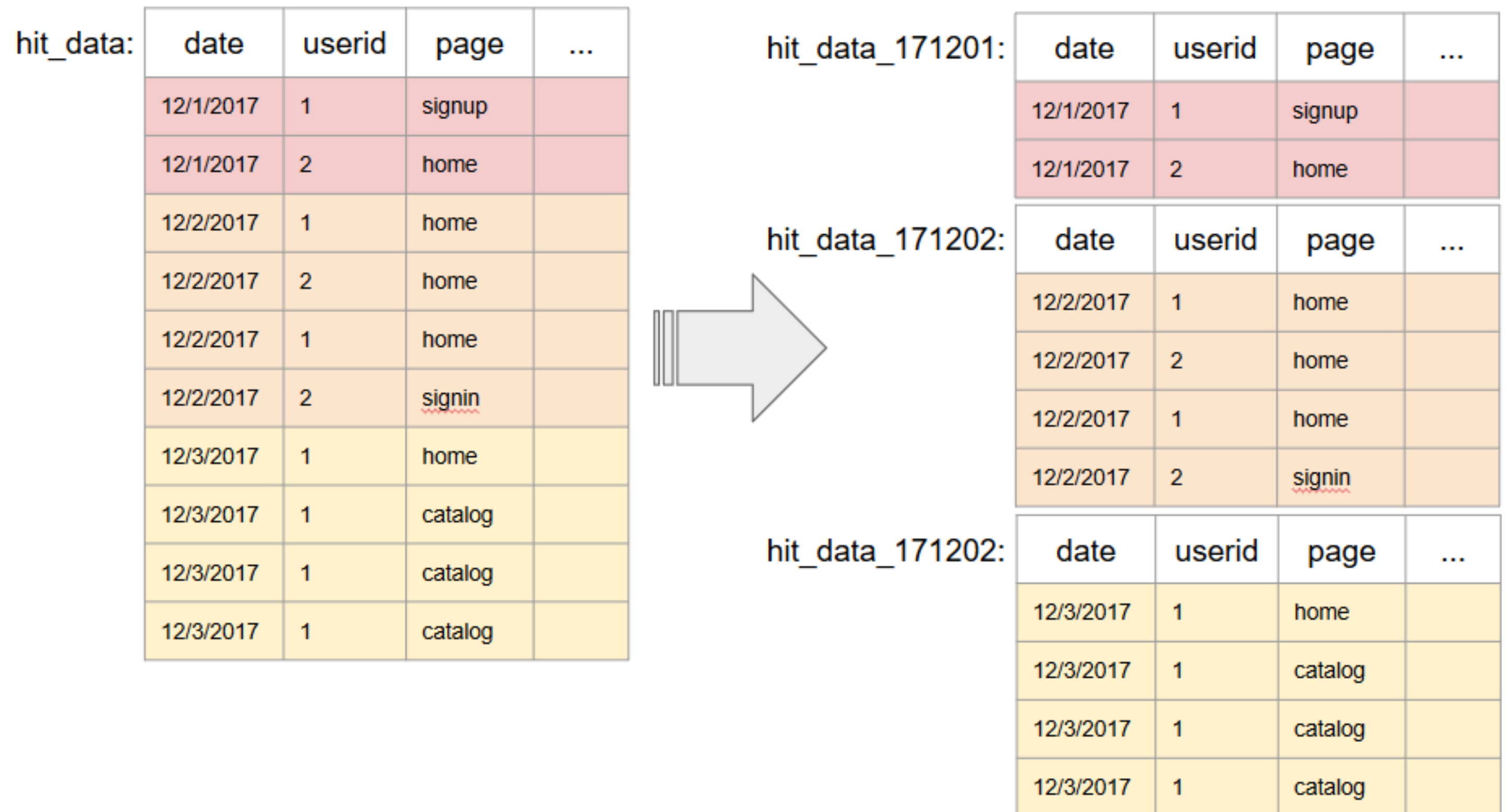


Партиционирование

Партиционирование — это метод разделения больших (исходя из количества записей, а не столбцов) таблиц на много маленьких. И желательно, чтобы это происходило прозрачным для приложения способом.

```
CREATE TABLE measurement (  
  city_id      int not null,  
  logdate     date not null,  
  peaktemp    int,  
  unitsales   int  
) PARTITION BY RANGE (logdate);
```

Partitioning Tables By Date



Партиции в диапазоне

```
CREATE TABLE sales (  
    id int,  
    date date,  
    amt decimal(10,2)  
)  
PARTITION BY RANGE (date)  
( START (date '2022-01-01') INCLUSIVE  
  END (date '2023-01-01') EXCLUSIVE  
  EVERY (INTERVAL '1 day'));
```

```
CREATE TABLE nrank (  
    id int,  
    rank int,  
    year int,  
    color char(1),  
    count int  
)  
PARTITION BY RANGE (year)  
( START (2012) END (2022) EVERY (1),  
  DEFAULT PARTITION extra);
```

```
CREATE TABLE sales (  
    id int,  
    date date,  
    amt decimal(10,2)  
)  
PARTITION BY RANGE (date)  
( PARTITION Jan22 START (date '2022-01-01') INCLUSIVE ,  
  PARTITION Feb22 START (date '2022-02-01') INCLUSIVE ,  
  PARTITION Mar22 START (date '2022-03-01') INCLUSIVE ,  
  PARTITION Apr22 START (date '2022-04-01') INCLUSIVE ,  
  PARTITION May22 START (date '2022-05-01') INCLUSIVE ,  
  PARTITION Jun22 START (date '2022-06-01') INCLUSIVE ,  
  PARTITION Jul22 START (date '2022-07-01') INCLUSIVE ,  
  PARTITION Aug22 START (date '2022-08-01') INCLUSIVE ,  
  PARTITION Sep22 START (date '2022-09-01') INCLUSIVE ,  
  PARTITION Oct22 START (date '2022-10-01') INCLUSIVE ,  
  PARTITION Nov22 START (date '2022-11-01') INCLUSIVE ,  
  PARTITION Dec22 START (date '2022-12-01') INCLUSIVE  
    END (date '2023-01-01') EXCLUSIVE);
```

Партиции по списку

```
CREATE TABLE rank (  
    id int,  
    rank int,  
    year int,  
    color char(1),  
    count int  
)  
PARTITION BY LIST (color)  
( PARTITION red VALUES ('r'),  
  PARTITION blue VALUES ('b'),  
  PARTITION green VALUES ('g'),  
  DEFAULT PARTITION other );
```

Таблица, партиционированная по списку, может использовать столбец любого типа данных, который допускает сравнения на равенство.

При классическом синтаксисе партиция по списку может иметь составной ключ раздела.

- Для разделов списка необходимо объявить раздел для каждого значения списка;
- При отсутствии дефолтной партиции невозможно добавление в таблицу данных, для которых не подходит ни одна партиция.

Многоуровневые партиции

В примере таблица sales партиционируется по year, затем по month, затем по region.

Условия SUBPARTITION TEMPLATE гарантируют, что каждый годовой раздел будет иметь одинаковую структуру подразделов.

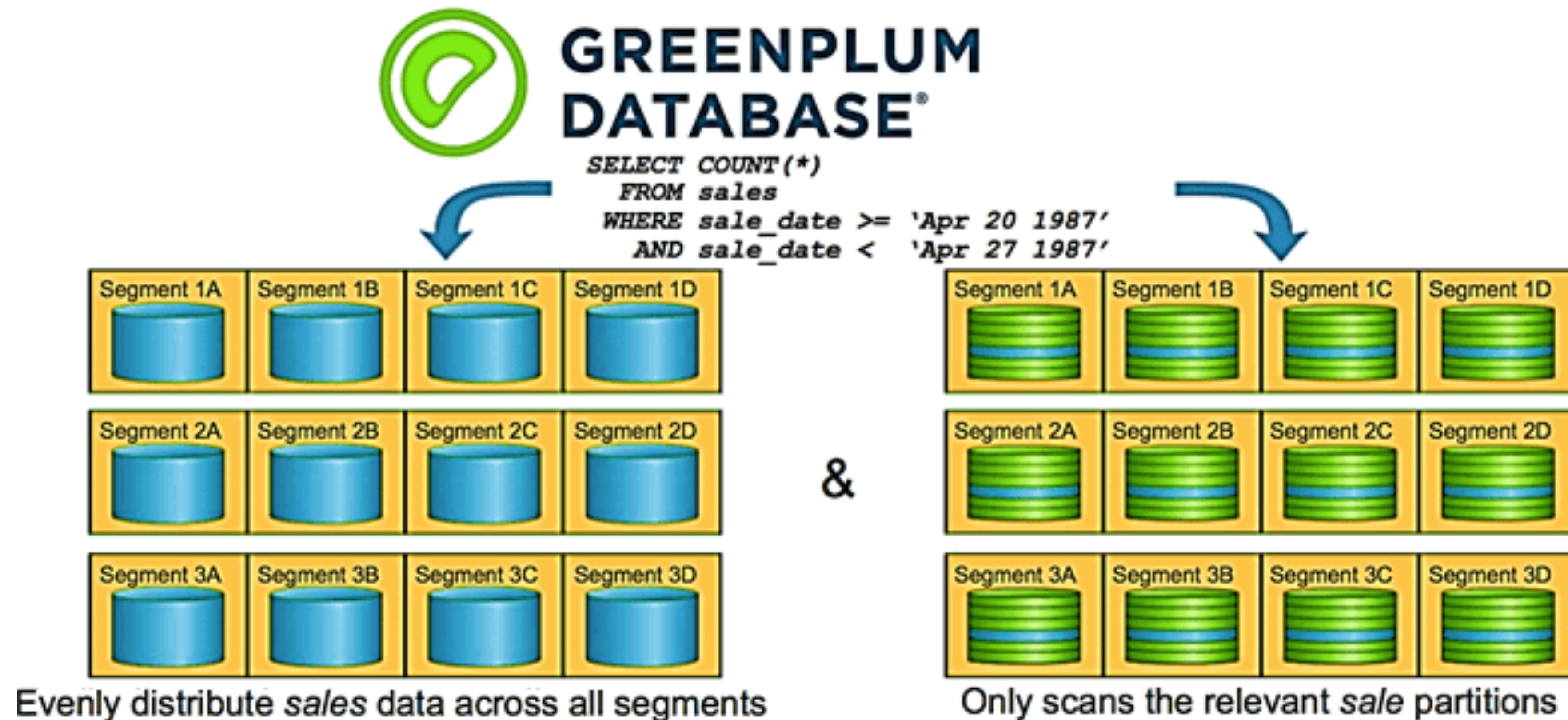
В примере на каждом уровне иерархии обязательно объявляется раздел DEFAULT

При создании многоуровневых разделов на основе диапазонов легко создать большое количество подразделов, некоторые из которых будут содержать мало данных или не будут содержать их вовсе. Это может привести к увеличению количества записей в системных таблицах, что, в свою очередь, увеличит время и объём памяти, необходимые для оптимизации и выполнения запросов.

```
CREATE TABLE p3_sales (  
    id int,  
    year int,  
    month int,  
    day int,  
    region text  
)  
PARTITION BY RANGE (year)  
    SUBPARTITION BY RANGE (month)  
        SUBPARTITION TEMPLATE (  
            START (1) END (13) EVERY (1),  
            DEFAULT SUBPARTITION other_months )  
        SUBPARTITION BY LIST (region)  
            SUBPARTITION TEMPLATE (  
                SUBPARTITION usa VALUES ('usa'),  
                SUBPARTITION europe VALUES ('europe'),  
                SUBPARTITION asia VALUES ('asia'),  
                DEFAULT SUBPARTITION other_regions )  
( START (2002) END (2012) EVERY (1),  
    DEFAULT PARTITION outlying_years );
```

Сегментация (дистрибуция)

Для распределения данных по кластерам используется их сегментация (Segmentation). Задача разработчика — подобрать такой список полей и/или такую функцию (например, хэш-функцию), благодаря которым данные равномерно распределятся по нодам кластера.



Виды сегментации

DISTRIBUTED BY

Чтобы корректно распределить данные по первичным сегментам, для выбранных полей формируется хэш-значения, на основе которых и будет проходить распределение.

```
Create table t1 (  
    col1 integer,  
    col2 varchar(100)  
) with (appendonly=false)  
distributed by (col1);
```

DISTRIBUTED REPLICATED

Зачастую используется для небольших таблиц (к примеру, словарей). Содержимое таблицы дублируется на каждый сегмент.

```
Create table t1 (  
    col1 integer,  
    col2 varchar(100)  
) with (appendonly=false)  
distributed replicated;
```

DISTRIBUTED RANDOMLY

Это – приблизительно равномерное распределение данных между сегментами. Данное распределение используется, если не подходит ни одно из двух предыдущих.

```
Create table t1 (  
    col1 integer,  
    col2 varchar(100)  
) with (appendonly=false)  
distributed randomly;
```